

Product Requirements Document (PRD)

BlogHub – Modern Blogging Platform (MVP)

Version: 1.0

Duration: 1 Week

Author: Sabbir Sharif

Role: Backend-Focused Portfolio Project

1. Product Overview

Product Name

BlogHub

Product Vision

BlogHub is a modern blogging platform that allows users to publish, manage, and discover articles. The project focuses on demonstrating production-ready backend engineering practices including authentication, authorization, REST API design, database modeling, validation, security, pagination, and deployment.

This project is intended to showcase backend development skills for junior software engineering positions rather than compete with full-scale blogging platforms like Medium.

2. Goals

Primary Goals

- Build a production-quality REST API
 - Demonstrate clean backend architecture
 - Practice Spring Boot best practices
 - Learn real-world database design
 - Deploy a cloud-hosted application
 - Create a strong portfolio project
-

3. Target Users

Guest Users

Can

- Browse published blogs
- Search blogs
- View categories
- Read articles

Cannot

- Create posts
 - Comment
 - Like
 - Bookmark
-

Registered Users

Can

- Create blogs
 - Save drafts
 - Publish blogs
 - Edit own blogs
 - Delete own blogs
 - Comment
 - Like blogs
 - Bookmark blogs
 - Manage profile
-

Admin

Can

- Manage all users
 - Delete inappropriate posts
 - Manage categories
 - View platform statistics
-

4. Functional Requirements

4.1 Authentication

User Registration

Users can register using

- Name
- Email
- Password

Validation

- Unique email
 - Strong password
 - Required fields
-

Login

Users authenticate using

- Email
- Password

Authentication uses

- JWT Access Token
 - Refresh Token
-

Logout

Invalidate refresh token.

Authorization

Two roles

- USER
 - ADMIN
-

4.2 User Profile

Each user has

- Name
- Username
- Email
- Bio
- Profile Image

- Created Date

Users can

- View profile
 - Edit profile
 - Change password
-

4.3 Blog Management

Users can

Create

Edit

Delete

Publish

Archive

Save as Draft

Each blog contains

- Title
 - Slug
 - Content (Markdown)
 - Excerpt
 - Cover Image
 - Status
 - Author
 - Category
 - Tags
 - Reading Time
 - Created Date
 - Updated Date
-

4.4 Categories

Admin manages categories.

Users select one category while publishing.

Examples

- Java
 - Spring Boot
 - Docker
 - Database
 - AI
 - Career
-

4.5 Tags

Users may assign multiple tags.

Examples

- spring
 - java
 - backend
 - jwt
 - docker
-

4.6 Comments

Authenticated users can

- Add comments
- Edit own comments
- Delete own comments

Admins can remove any comment.

4.7 Likes

Users may like a post.

Rules

- One like per user
- User may unlike

4.8 Bookmarks

Users may bookmark articles.

Users have a personal bookmark collection.

4.9 Search

Search by

- Title
 - Content
 - Tag
-

4.10 Filtering

Users can filter blogs by

- Category
 - Author
 - Tag
 - Status
-

4.11 Pagination

Every listing endpoint supports

- Page
 - Size
 - Sort
 - Direction
-

4.12 Slug URLs

Each published article has a unique slug.

Example

`/blogs/build-rest-api-with-spring-boot`

4.13 Reading Time

Automatically calculate estimated reading time based on article length.

4.14 View Count

Increase view count whenever an article is opened.

4.15 API Documentation

Swagger/OpenAPI documentation.

5. Non-Functional Requirements

Performance

- API response under 300 ms for normal requests
 - Support pagination for all listing endpoints
-

Security

- Password hashing using BCrypt
 - JWT Authentication
 - Role-based authorization
 - Input validation
 - Global exception handling
-

Reliability

- Graceful error handling
 - Consistent API responses
 - Logging
-

Maintainability

- Layered Architecture
- DTO pattern

- Service abstraction
 - Repository pattern
 - Clean package structure
-

6. Technology Stack

Backend

- Java 21
- Spring Boot 3

Database

- PostgreSQL

Security

- Spring Security
- JWT

ORM

- Spring Data JPA
- Hibernate

Documentation

- Swagger/OpenAPI

Validation

- Hibernate Validator

Utilities

- Lombok
- MapStruct (optional)

Database Migration

- Flyway

Deployment

- Docker
- Docker Compose
- Render/Railway

Version Control

- Git
 - GitHub
-

7. Core Database Entities

User



Role

RefreshToken

Post

Category

Tag

Comment

Like

Bookmark

8. High-Level Entity Relationships

User



├─ Posts (1:N)
├─ Comments (1:N)
├─ Likes (1:N)
└─ Bookmarks (1:N)

Post

├─ Category (N:1)
├─ Tags (N:M)
├─ Comments (1:N)
├─ Likes (1:N)
└─ Author (N:1)

Role

└─ Users (1:N)

9. Success Criteria

The project is considered complete when

- User authentication works
- CRUD operations are fully functional

- Authorization is enforced
 - Search and pagination work
 - API documentation is available
 - Dockerized application runs successfully
 - Application is deployed
 - GitHub repository includes a professional README
-

10. Future Enhancements (Out of Scope)

- Email verification
 - Password reset
 - Rich text editor
 - Follow users
 - Notifications
 - Trending posts
 - Full-text search (Elasticsearch)
 - Redis caching
 - File storage using AWS S3
 - Analytics dashboard
 - Admin moderation panel
 - CI/CD pipeline
 - OAuth login (Google/GitHub)
 - Real-time notifications
 - GraphQL API
-

MVP Scope (One-Week Delivery)

For the one-week timeline, the project will include:

- JWT Authentication & Authorization
- User Profile Management
- Blog CRUD (Draft/Publish/Archive)
- Categories & Tags
- Search, Filtering, Pagination & Sorting
- Comments
- Likes & Bookmarks
- Slug-based URLs
- Reading Time & View Count
- Global Exception Handling & Validation

- Swagger/OpenAPI Documentation
- Flyway Database Migrations
- Docker & Docker Compose
- PostgreSQL Database
- Deployment (Render/Railway)
- Professional README with API documentation

This scope is realistic for one week while remaining comprehensive enough to demonstrate the backend skills expected of a junior Java/Spring Boot developer.